Module 4

UVM Components

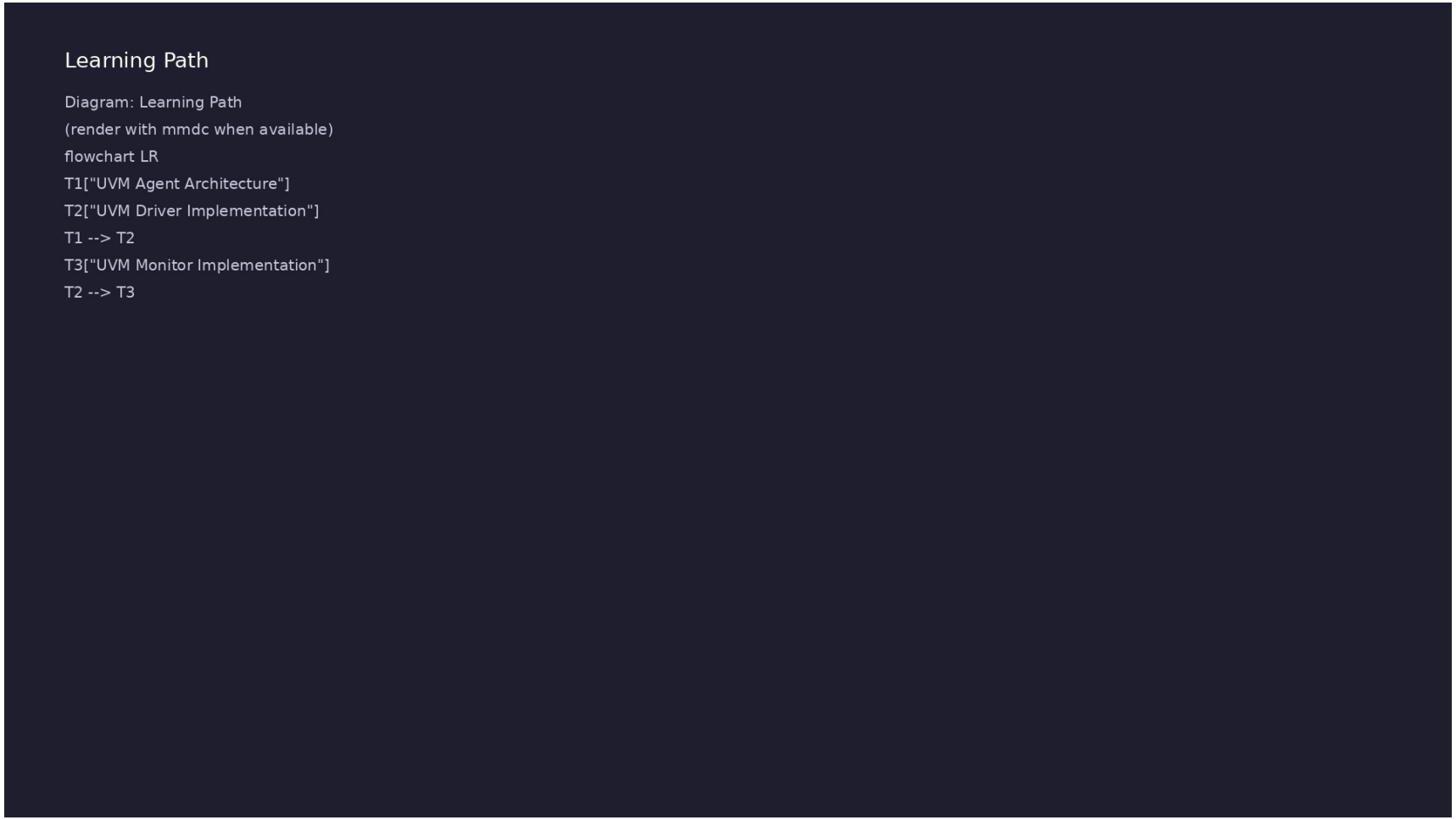
Learning objectives

- Build complete UVM agents with driver, monitor, and sequencer

Prerequisites

- See module README

Learning path



Build complete UVM agents with driver, monitor, and sequencer

Overview

- This module covers the core UVM components used to build verification environments. You'll learn ho…

How to learn this module

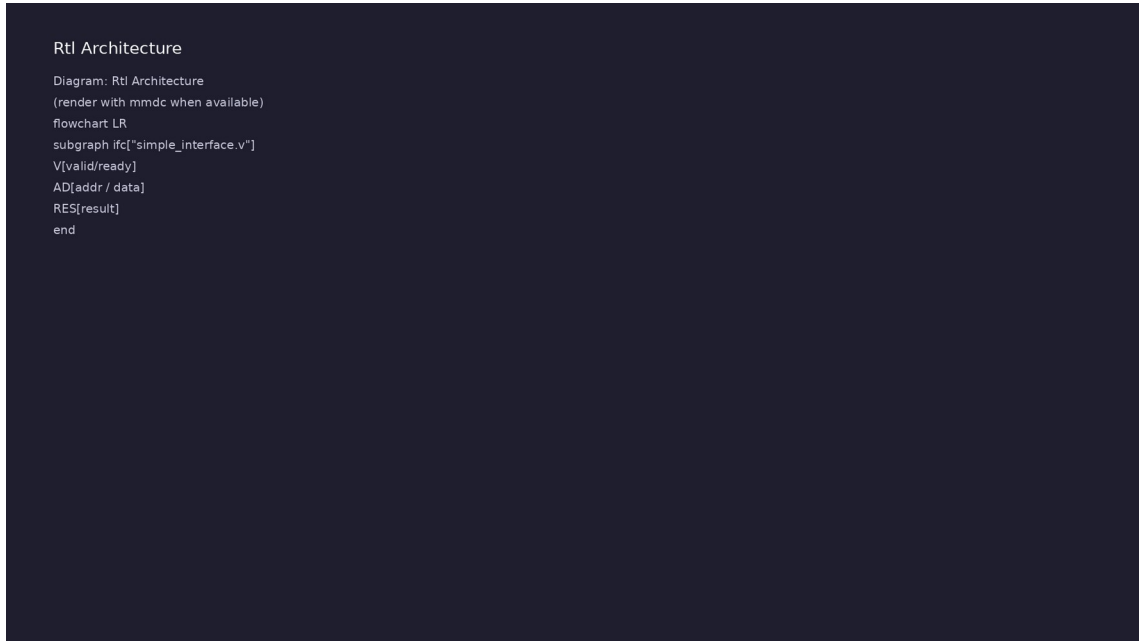
Suggested learning path

- Module 3 UVM hierarchy and phases must be comfortable — agents bundle components you studied separa…
- Study `module4/dut/interfaces/simple_interface.v` valid/ready handshake before driver labs
- Run examples in order: transactions → driver → monitor → sequencer → scoreboard → TLM → agents
- Read `InterfaceTransaction` field definitions and `copy`/`compare` before scoreboard integration
- Integrated capstone is `test_complete_agent.py` — do not skip per-component examples

Design architecture

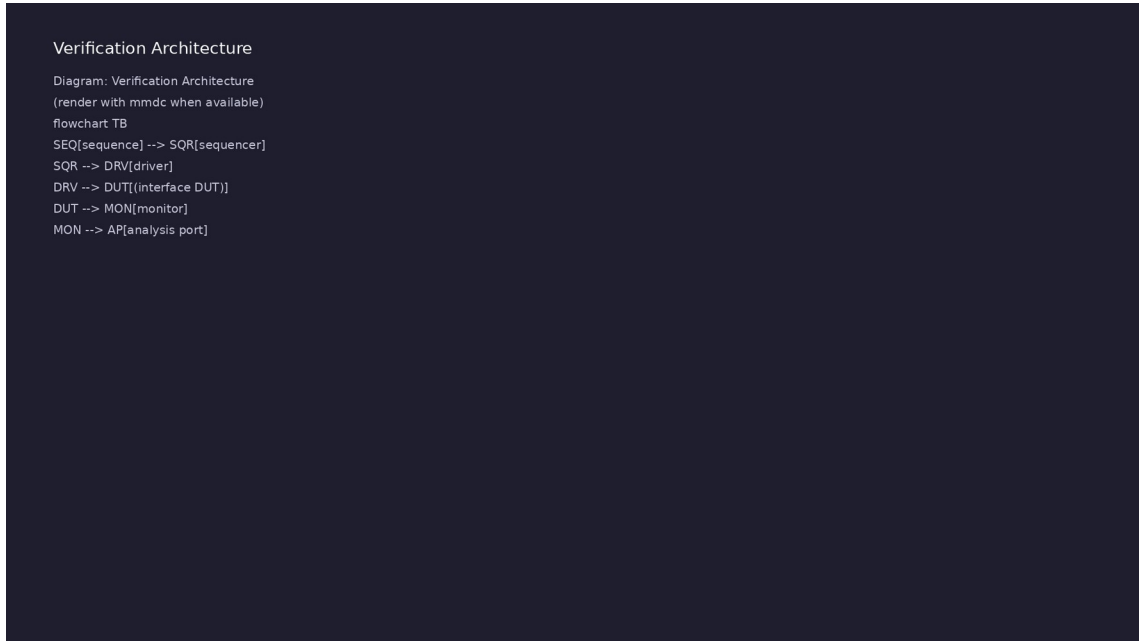
1. DUT and interface protocol

- `module4/dut/interfaces/simple_interface.v` — valid/ready handshake, address/data/result buses
- Clocked, resettable interface models streaming transactions
- Agent maps one transaction type to one interface beat



2. Agent-centric UVM architecture

- Agent bundles driver, monitor, sequencer; env holds agent + scoreboard
- TLM analysis ports connect monitor → scoreboard (and optional coverage)
- `InterfaceTransaction` defines fields, `copy`/`compare`, and constraints
- Complete path: sequence → sequencer → driver → DUT → monitor → scoreboard



3. Execution pipeline

- Build: Verilator compiles `simple_interface.v`; env builds agent (driver, monitor, sequencer) and s...
- Connect: monitor `ap` → scoreboard `exp`/`act` analysis imports; sequencer ↔ driver TLM ports
- Sim: sequence items drive valid/ready beats; monitor reconstructs transactions on completed handsha...
- Check: scoreboard `compare` on monitored vs predicted streams; UVM report summarizes mismatches

4. Component responsibilities

- Driver: pull items from sequencer, drive pins per protocol
- Monitor: passive observation, broadcast transactions on analysis port
- Sequencer: arbitration and sequence execution
- Scoreboard: expected vs actual from monitor streams

Interface — valid/ready handshake

```
/**
 * Module 4: Simple Interface
 *
 * A simple interface for UVM component testing.
 *
 * Ports:
 *   clk:    Clock signal
 *   rst_n:  Active-low reset
 *   valid:  Valid signal
 *   ready:  Ready signal
 *   data:   Data bus
 *   address: Address bus
 */

module simple_interface (
    input  wire       clk,
# ...
```

Agent-centric env wiring

```
"""
Module 4 Test: Complete Agent Test
Complete UVM testbench with driver, monitor, sequencer, and scoreboard.
"""


import cocotb
from cocotb.clock import Clock
from cocotb.triggers import Timer, RisingEdge
from pyuvm import *
# Explicitly import uvm_seq_item_pull_port - it may not be exported by
from pyuvm import *
# Try multiple possible import paths
_uvm_seq_item_pull_port = None
try:
    # First try: check if it's in the namespace after from pyuvm import
*
    _uvm_seq_item_pull_port = globals()['uvm_seq_item_pull_port']
```

Key files to study

Open these in the repo

- `module4/dut/interfaces/simple_interface.v` — valid/ready streaming interface with addr/data/result
- `module4/examples/transactions/transaction_example.py` — sequence item fields and compare/copy
- `module4/examples/drivers/driver_example.py` — sequencer pull loop and pin wiggling
- `module4/examples/monitors/monitor_example.py` — passive sampling and analysis port broadcast
- `module4/tests/pyuvm_tests/test_complete_agent.py` — end-to-end agent, env, scoreboard integration
- `scripts/module4.sh` — `--drivers` … `--agents` example flags and `--pyuvm-tests`

Verification & testing methods

1. Integration testing method

- `test_complete_agent.py` — end-to-end agent, env, and test class
- Run: `./scripts/module4.sh --pyuvm-tests` or `make TEST=test_complete_agent`
- Per-component examples (`driver_example.py`, ...) isolate behavior before integration

2. Checking strategy

- Scoreboard compares monitored results to predicted model or golden vectors
- Analysis port fan-out allows multiple checkers without changing monitor
- Sequences provide repeatable stimulus; random/constrained extensions in exercises

3. Step-by-step lab execution

- 1. Transactions: `./scripts/module4.sh --transactions`
- 2. Driver/monitor/sequencer: `./scripts/module4.sh --drivers --monitors --sequencers`
- 3. Scoreboard/TLM: `./scripts/module4.sh --scoreboards --tlm`
- 4. Agent assembly: `./scripts/module4.sh --agents`
- 5. Capstone: `./scripts/module4.sh --pyuvm-tests` — confirm scoreboard clean and UVM TEST PASSED

4. Closure

- Self-check via `./scripts/module4.sh --pyuvm-tests`;
  demo screenshots per EXAMPLES.md section
- Assessment: drivers, monitors, sequencers, agents,
  scoreboards, TLM connections
- You should trace one transaction from sequence item
  through to scoreboard compare

Syllabus topics

1. UVM Agent Architecture (1/3)

- Agent Overview
- What is an agent?
- Agent components
- Agent purpose
- Agent types
- Active vs Passive Agents

1. UVM Agent Architecture (2/3)

- Active agents (driver + sequencer + monitor)
- Passive agents (monitor only)
- When to use each
- Agent configuration
- Agent Structure
- Driver component

1. UVM Agent Architecture (3/3)

- Monitor component
- Sequencer component
- Agent container

2. UVM Driver Implementation (1/4)

- Driver Overview
- Driver purpose
- Driver responsibilities
- Driver interface
- Driver lifecycle
- Driver Implementation

2. UVM Driver Implementation (2/4)

- Inheriting from `uvm_driver`
- `run_phase()` implementation
- Transaction reception
- Signal driving
- Driver-Sequencer Communication
- `seq_item_port` interface

2. UVM Driver Implementation (3/4)

- `get_next_item()`
- `item_done()`
- Transaction flow
- Signal-Level Driving
- DUT signal access
- Signal value assignment

2. UVM Driver Implementation (4/4)

- Timing control
- Protocol implementation

3. UVM Monitor Implementation (1/4)

- Monitor Overview
- Monitor purpose
- Monitor responsibilities
- Monitor types
- Monitor lifecycle
- Monitor Implementation

3. UVM Monitor Implementation (2/4)

- Inheriting from `uvm_monitor`
- `run_phase()` implementation
- Signal sampling
- Transaction creation
- Analysis Ports
- Analysis port purpose

3. UVM Monitor Implementation (3/4)

- Creating analysis ports
- Writing to analysis ports
- Analysis port connections
- Transaction Creation
- Sampling signals
- Creating transaction objects

3. UVM Monitor Implementation (4/4)

- Populating transaction fields
- Broadcasting transactions

4. UVM Sequencer and Sequences (1/5)

- Sequencer Overview
- Sequencer purpose
- Sequencer responsibilities
- Sequencer types
- Sequencer lifecycle
- Sequencer Implementation

4. UVM Sequencer and Sequences (2/5)

- Inheriting from `uvm_sequencer`
- Default sequencer usage
- Custom sequencer features
- Sequencer configuration
- Sequence Items
- `uvm_sequence_item` base class

4. UVM Sequencer and Sequences (3/5)

- Transaction definition
- Transaction fields
- Transaction methods
- Sequence Basics
- `uvm_sequence` base class
- `body()` method

4. UVM Sequencer and Sequences (4/5)

- Sequence execution
- Sequence lifecycle
- Sequence Operations
- `start_item()`
- `finish_item()`
- `wait_for_grant()`

4. UVM Sequencer and Sequences (5/5)

- Transaction creation

5. TLM (Transaction-Level Modeling) - Complete Coverage (1/11)

- TLM Overview
- What is TLM?
- TLM benefits
- TLM abstraction levels
- TLM communication patterns
- TLM Interface Types

5. TLM (Transaction-Level Modeling) - Complete Coverage (2/11)

- `put` interface - Unidirectional blocking put
- `get` interface - Unidirectional blocking get
- `peek` interface - Unidirectional non-blocking peek
- `transport` interface - Bidirectional blocking transport
- Interface characteristics and use cases
- TLM Port Types

5. TLM (Transaction-Level Modeling) - Complete Coverage (3/11)

- `uvm_put_port` - Put port
- `uvm_get_port` - Get port
- `uvm_peek_port` - Peek port
- `uvm_transport_port` - Transport port
- Port vs export vs implementation
- TLM Export Types

5. TLM (Transaction-Level Modeling) - Complete Coverage (4/11)

- `uvm_put_export` - Put export
- `uvm_get_export` - Get export
- `uvm_peek_export` - Peek export
- `uvm_transport_export` - Transport export
- Export usage patterns
- TLM Implementation Types

5. TLM (Transaction-Level Modeling) - Complete Coverage (5/11)

- `uvm_put_imp` - Put implementation
- `uvm_get_imp` - Get implementation
- `uvm_peek_imp` - Peek implementation
- `uvm_transport_imp` - Transport implementation
- Implementation requirements
- Analysis Ports and Exports (Special TLM Type)

5. TLM (Transaction-Level Modeling) - Complete Coverage (6/11)

- Analysis port concept
- Publisher-subscriber pattern
- Broadcast communication
- One-to-many connections
- `uvm_analysis_port` - Analysis port
- `uvm_analysis_export` - Analysis export

5. TLM (Transaction-Level Modeling) - Complete Coverage (7/11)

- `uvm_analysis_imp` - Analysis implementation
- Connection patterns
- TLM FIFOs
- `uvm_tlm_fifo` - TLM FIFO
- FIFO purpose and usage
- FIFO capacity and blocking behavior

5. TLM (Transaction-Level Modeling) - Complete Coverage (8/11)

- FIFO connection patterns
- When to use FIFOs
- TLM Connection Patterns
- Direct connections (port to export)
- FIFO connections (port to FIFO to export)
- Multi-port connections

5. TLM (Transaction-Level Modeling) - Complete Coverage (9/11)

- Hierarchical connections
- Connection best practices
- TLM Usage Patterns
- Producer-consumer pattern
- Request-response pattern
- Broadcast pattern

5. TLM (Transaction-Level Modeling) - Complete Coverage (10/11)

- Pipeline pattern
- TLM vs analysis ports
- TLM Implementation Examples
- Using put/get interfaces
- Using transport interface
- Using TLM FIFOs

5. TLM (Transaction-Level Modeling) - Complete Coverage (11/11)

- Combining TLM types
- TLM debugging

6. Scoreboard Implementation (1/4)

- Scoreboard Overview
- Scoreboard purpose
- Scoreboard types
- Scoreboard responsibilities
- Scoreboard lifecycle
- Scoreboard Implementation

6. Scoreboard Implementation (2/4)

- Inheriting from `uvm_component`
- Analysis port connections
- Transaction storage
- Comparison logic
- Scoreboard Patterns
- Reference model comparison

6. Scoreboard Implementation (3/4)

- Expected vs actual
- Transaction matching
- Error reporting
- Advanced Scoreboards
- Multi-channel scoreboards
- Time-based matching

6. Scoreboard Implementation (4/4)

- Complex comparison logic
- Performance optimization

7. Transaction-Level Modeling (1/3)

- Transaction Concepts
- What are transactions?
- Transaction abstraction
- Transaction fields
- Transaction methods
- Transaction Design

7. Transaction-Level Modeling (2/3)

- Transaction class structure
- Field definition
- Constraint definition
- Method implementation
- Transaction Operations
- Transaction creation

7. Transaction-Level Modeling (3/3)

- Transaction copying
- Transaction comparison
- Transaction conversion

8. Complete Agent Example (1/3)

- Agent Structure
- Agent class definition
- Component instantiation
- Component connections
- Agent configuration
- Agent Build Phase

8. Complete Agent Example (2/3)

- Component creation
- Configuration application
- Active/passive selection
- Agent Connect Phase
- Driver-sequencer connection
- Monitor analysis port

8. Complete Agent Example (3/3)

- External connections

9. Sequence Libraries (1/2)

- Sequence Organization
- Base sequences
- Derived sequences
- Sequence libraries
- Sequence reuse
- Common Sequences

9. Sequence Libraries (2/2)

- Simple sequences
- Random sequences
- Constrained sequences
- Layered sequences

10. Agent Integration (1/2)

- Environment Integration
- Adding agents to environment
- Agent configuration
- Agent connections
- Agent coordination
- Test Integration

10. Agent Integration (2/2)

- Agent instantiation
- Sequence execution
- Test coordination
- Result checking

Command reference highlights

Per-component examples

- `./scripts/module4.sh --transactions --drivers --monitors` — stimulus and observation building bloc…
- `./scripts/module4.sh --sequencers --scoreboards --tlm` — sequence flow and analysis connections
- `./scripts/module4.sh --agents` — complete agent wiring before capstone test

Integration test

- `./scripts/module4.sh --pyuvm-tests` — run `test_complete_agent` via orchestrator
- `cd module4/tests/pyuvm_tests && make SIM=verilator TEST=test_complete_agent`
- `./scripts/module4.sh --skip-examples --pyuvm-tests` — capstone only when components are understood

Debug paths

- Run individual example Make under `module4/examples/<component>/` to isolate failures
- Increase UVM verbosity to trace sequence → driver → monitor transaction paths
- Verilator waveforms on interface valid/ready when handshake debug is needed

Hands-on examples

Module 4 orchestrator

```
$ ./scripts/module4.sh --help
```

Terminal: module4_check

```
[0;36m==========================================[0m
[0;36mModule 4: UVM Components[0m
[0;36m==========================================[0m

Usage: ./scripts/module4.sh [OPTIONS]

Module 4: UVM Components
This script runs examples and tests for Module 4.

OPTIONS:
    Examples:
        --drivers          Run driver examples
        --monitors         Run monitor examples
        --sequencers       Run sequencer and sequence examples
        --tlm              Run TLM examples
        --scoreboards      Run scoreboard examples
        --transactions     Run transaction examples
        --agents           Run complete agent examples
        --all-examples     Run all examples (default)
        --skip-examples    Skip all examples

    Tests:
        --pyuvm-tests      Run pyuvm tests

    Environment:
        --venv DIR         Virtual environment directory (default: .venv)
        --no-venv          Don't use virtual environment
        --sim SIMULATOR    Simulator to use (default: verilator)

    Other:
        --help, -h         Show this help message

EXAMPLES:
    # Run all examples
    ./scripts/module4.sh
```

Exercise scaffold

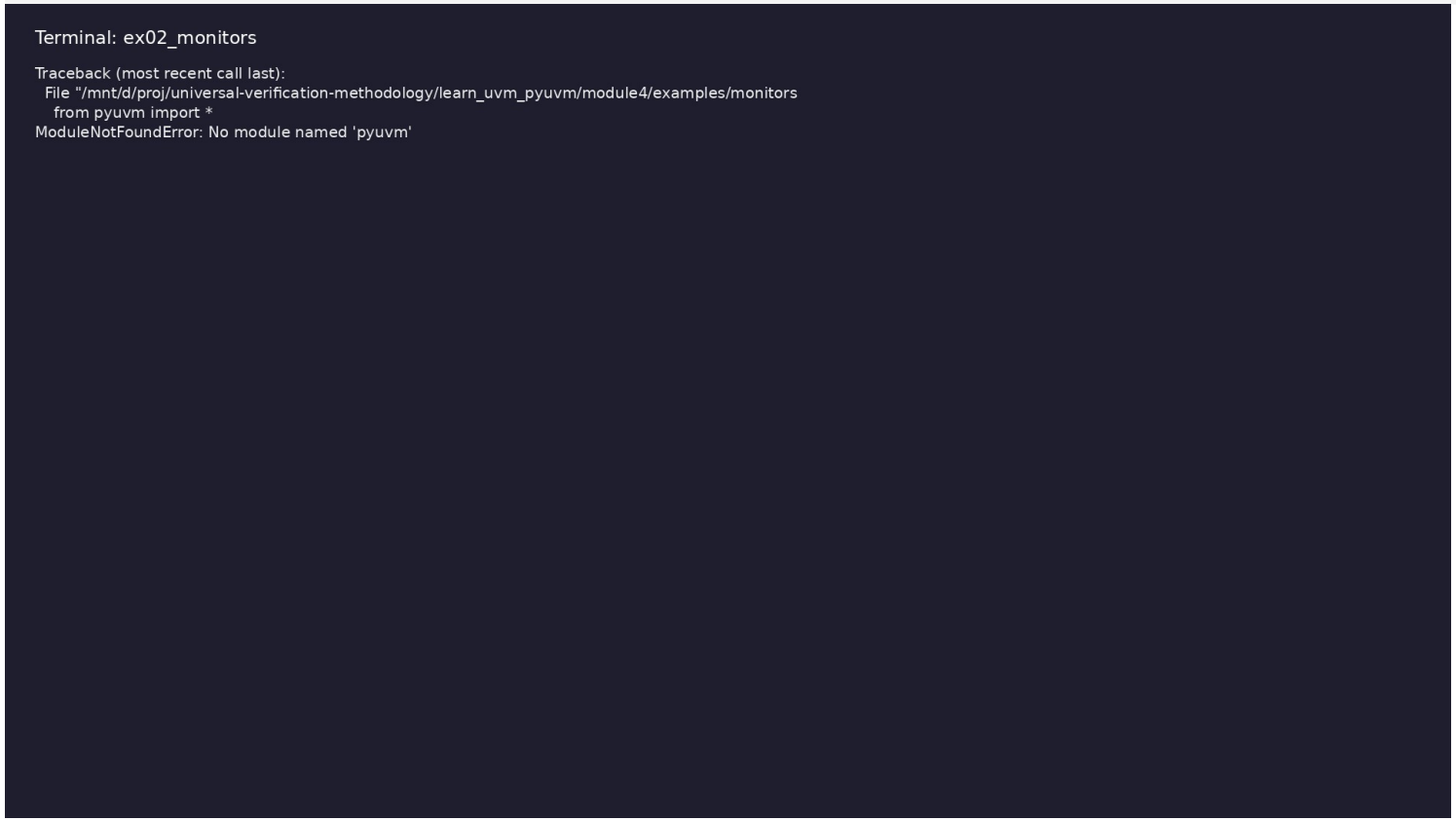
```
./scripts/module4.sh --scaffold
```

Demo: Driver Implementation

```
$ .venv/bin/python module4/examples/drivers/driver_example.py
```

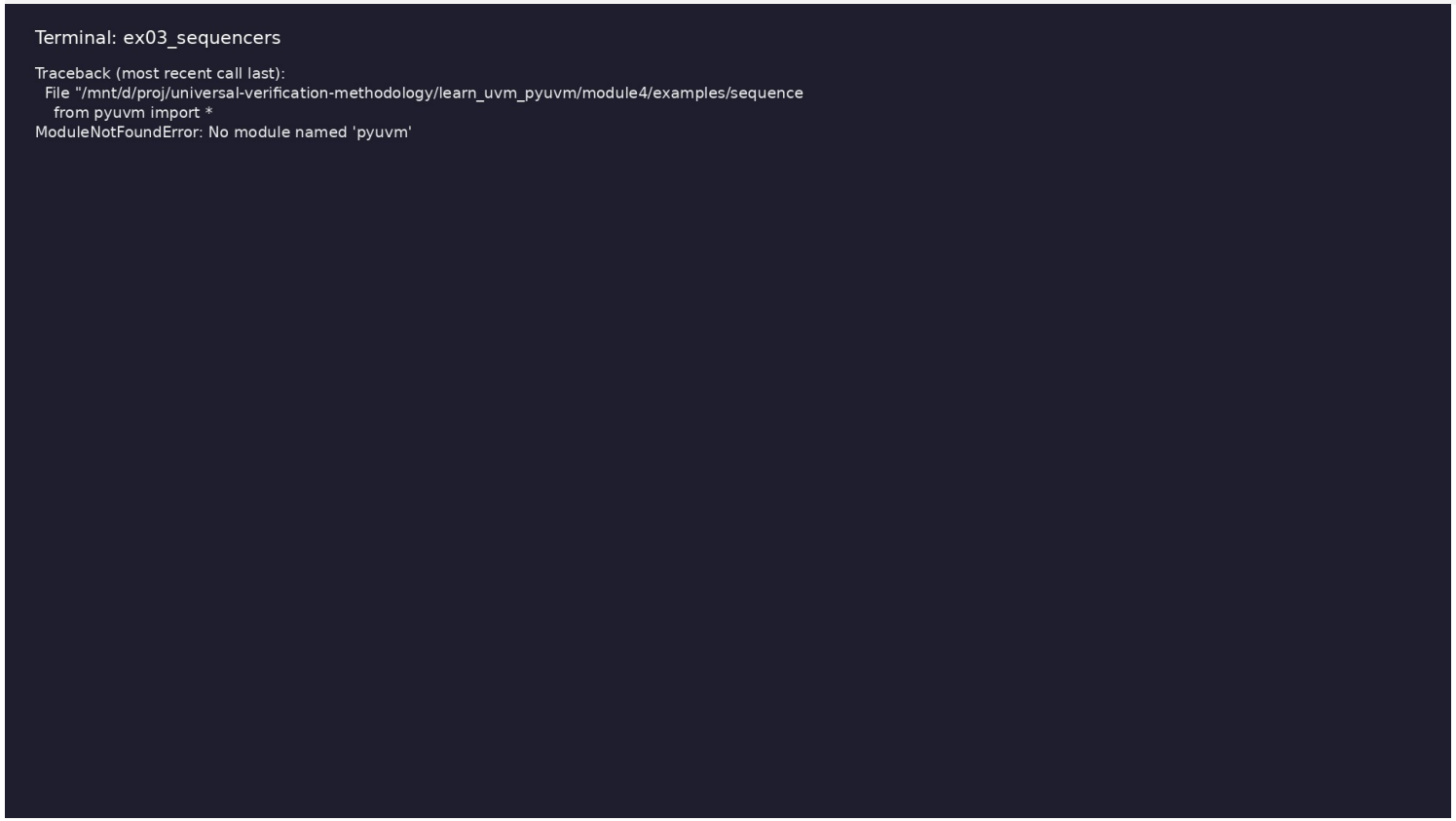
Demo: Monitor Implementation

```
$ .venv/bin/python module4/examples/monitors/monitor_example.py
```



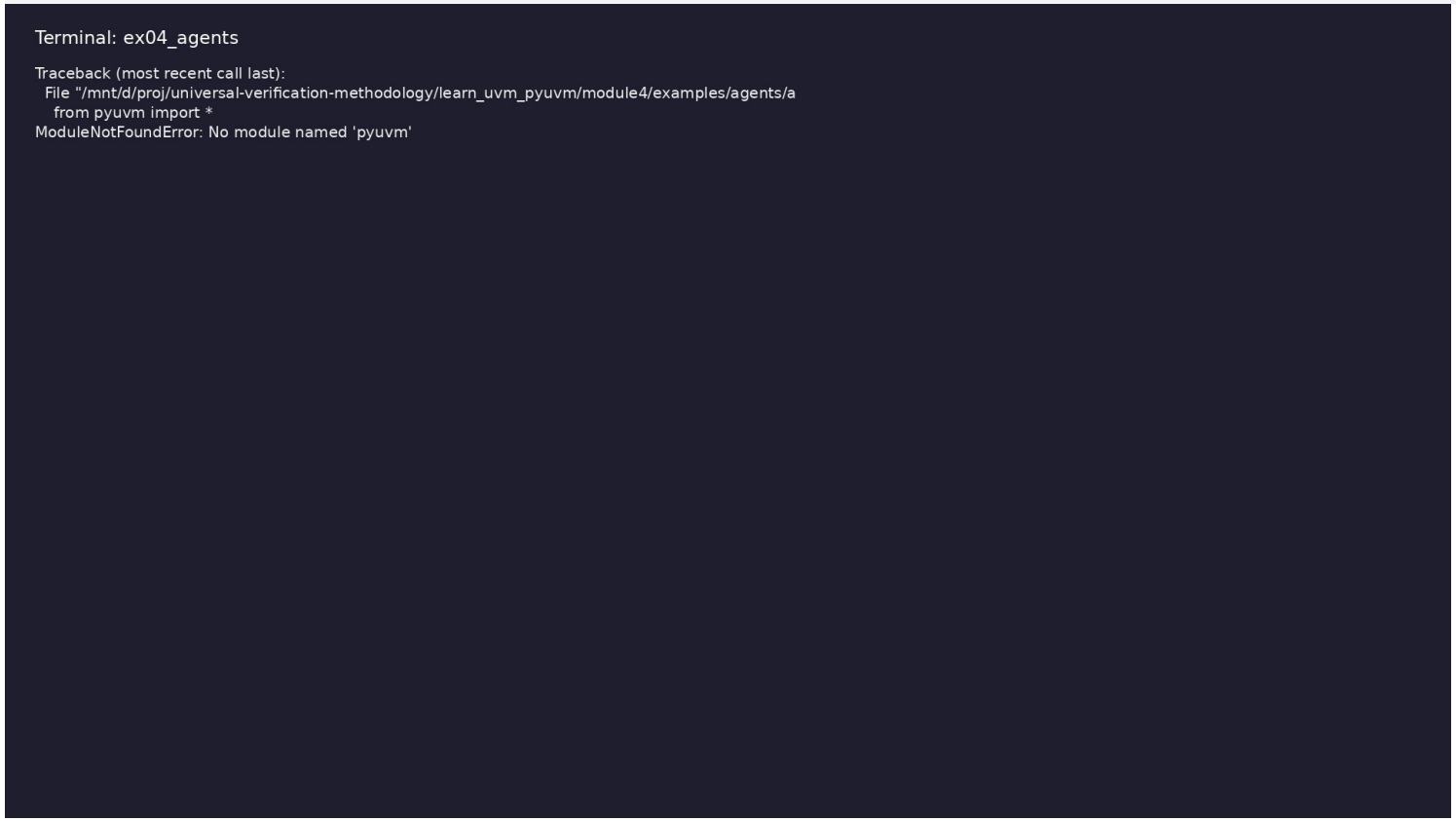
Demo: Sequencer and Sequences

```
$ .venv/bin/python module4/examples/sequencers/sequencer_example.py
```



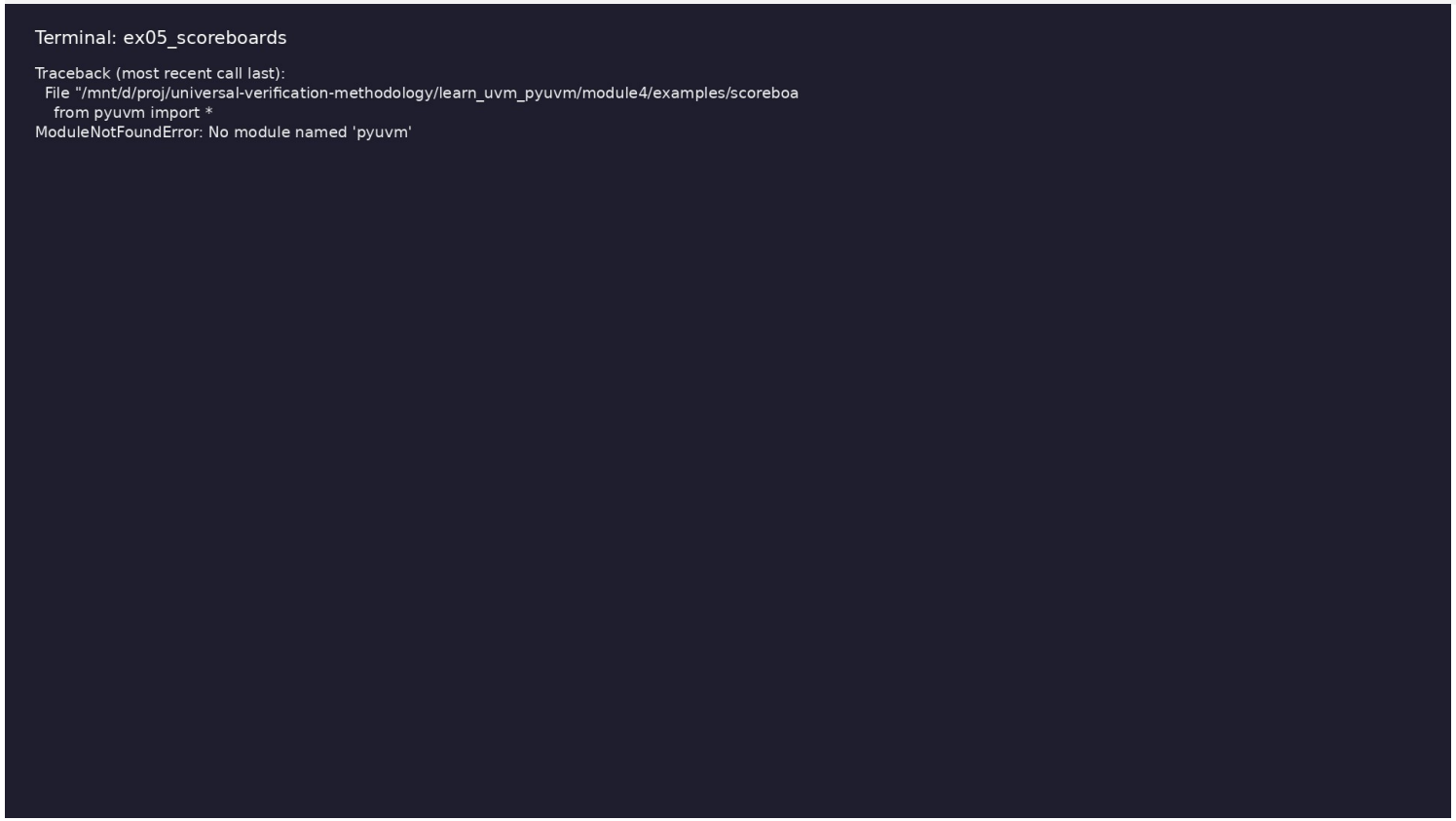
Demo: Complete Agent

```
$ .venv/bin/python module4/examples/agents/agent_example.py
```



Demo: Scoreboard Implementation

```
$ .venv/bin/python module4/examples/scoreboards/scoreboard_example.py
```



Demo: TLM Communication

```
$ .venv/bin/python module4/examples/tlm/tlm_example.py
```

Demo: Transaction Modeling

```
$ .venv/bin/python
      module4/examples/transactions/transaction_example.py
```

Practice & assessment

What you should know (1/3)

- Understand agent architecture
- Implement UVM drivers
- Implement UVM monitors
- Implement sequencers and sequences
- Use analysis ports effectively
- Implement scoreboards

What you should know (2/3)

- Design transaction models
- Build complete agents
- Integrate agents into environments
- Execute sequences in tests
- Driver class
- Transaction reception

What you should know (3/3)

- Signal driving

Exercises

- Driver Implementation
- Monitor Implementation
- Sequence Creation
- Agent Building
- Scoreboard Implementation

Assessment checklist

- Understands agent architecture
- Can implement drivers
- Can implement monitors
- Can implement sequencers
- Can create sequences
- Can use analysis ports

Summary & next steps

- Build complete UVM agents with driver, monitor, and sequencer
- Complete module4/CHECKLIST.md
- Review module4/EXAMPLES.md and run each lab
- Next: Next module in course